PRODUCT REQUIREMENTS DOCUMENT (PRD)

Project Name: CoupleConnect Game Hub (2-Player LDR Board Games)

Document Status: Final / Ready for Development

1. Executive Summary

This document outlines the technical and functional specifications for a real-time, web-based multiplayer game hub designed exclusively for 2 players (specifically targeting LDR couples). The platform emphasizes zero-friction onboarding (no authentication/login required), utilizing an auto-generated alphanumeric room code system. The primary differentiators are the high-quality UI/UX animations and the integration of custom "Truth or Dare" mechanics into classic board games.

2. Technical Architecture & Stack

The system is divided into a decoupled Frontend and Backend to ensure scalability and real-time performance. The architecture leverages modern web standards.

- **Frontend Application:**
  - **Framework:** React (bootstrapped with Vite for fast HMR).
  - **Styling:** Tailwind CSS. Strict requirement for pixel-perfect layout and responsive mobile-first design.
  - **Animation:** Framer Motion for dice rolls, board transitions, card flipping, and modal overlays.
  - **Hosting:** Vercel (linked to custom domain).
- **Backend Service:**
  - **Runtime:** Node.js with Express.js.
  - **Real-time Engine:** Socket.io for bi-directional event emission (game state sync).
  - **Containerization:** Dockerized backend environment (using a `Dockerfile` based on Alpine Node image) to ensure environment consistency before deploying.
  - **Hosting:** Render or Railway (ideal for WebSockets).

3. Comprehensive Directory Structure

To ensure Claude generates a robust architecture without missing logic, the following exact folder and file structure must be adhered to.

3.1 Frontend Structure (React + Vite)

```
frontend/
├── index.html
├── package.json
├── tailwind.config.js
├── vite.config.js
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   ├── index.css
│   ├── socket.js            # Socket.io client instance initialization
│   ├── assets/
│   │   ├── sounds/          # Dice roll, piece move, card draw MP3s
│   │   └── images/
│   ├── components/
│   │   ├── ui/              # Reusable Tailwind components (Buttons, Modals)
│   │   ├── TruthDareCard.jsx # Framer motion animated 3D flip card
│   │   ├── DiceRenderer.jsx
│   │   └── ChatBox.jsx
│   ├── pages/
│   │   ├── LandingPage.jsx   # Create/Join room logic
│   │   ├── LobbyPage.jsx     # Waiting for Player 2
│   │   ├── MonopolyBoard.jsx
│   │   ├── SnakesLadders.jsx
│   │   ├── LudoBoard.jsx
│   │   └── ChessBoard.jsx
│   ├── store/               # Zustand or React Context for local state
│   │   └── gameStore.js
│   └── utils/
│       ├── truthDareData.js  # Array of T/D questions
│       └── gameLogic.js      # Helper functions (e.g., checking win conditions)
```

3.2 Backend Structure (Node.js + Socket.io)

```
backend/
├── package.json
├── server.js                # Entry point, Express & HTTP server init
├── Dockerfile               # Containerization setup for Render
├── .env                     # CORS origins, PORT configurations
├── src/
│   ├── socket/
│   │   ├── index.js         # Main socket connection handler
│   │   ├── roomHandlers.js  # Logic for create_room, join_room, disconnect
│   │   ├── gameHandlers.js  # Logic for move_piece, roll_dice, draw_card
│   │   └── stateManager.js  # In-memory store for active rooms (Map/Object)
│   └── utils/
│       └── rng.js           # Cryptographically secure random generators
```

4. Core User Flow & Matchmaking

1. **Initialization:** User lands on the frontend application. They are presented with a highly polished, animated landing screen.
2. **Room Creation:** Player 1 clicks "Create Room". The backend generates a random 6-character uppercase alphanumeric code (e.g., `A7X9BQ`). Player 1 is routed to the Lobby.
3. **Joining:** Player 1 shares the code via external chat. Player 2 enters the code in "Join Room".
4. **Validation:** The backend validates the code. If the room already has 2 players, the socket connection emits a `ROOM_FULL` error. If valid, both players are synced to the game selection state.
5. **Game Selection:** Either player can select the game (Snakes & Ladders, Ludo, Chess, Monopoly). Once selected, the board component mounts.

5. Game Mechanics & Custom Rules

5.1 Snakes & Ladders (LDR Edition)

- **Board:** 10x10 grid (100 tiles) arranged in a boustrophedon (snake-like) pattern.

- **Custom Mechanic (The 6-Step Rule):** Track the exact number of steps each player moves cumulatively. Every time a player's cumulative step counter hits a multiple of 6, a Socket event `TRIGGER_TRUTH_DARE` is fired.
- **Card Draw:** A modal pops up on both screens. The drawing player selects "Truth" or "Dare". The card flips (using Framer Motion) revealing the prompt.
- **Progression:** The other player must click a "Done / Verified" button on their UI to allow the game to continue.

5.2 Monopoly (LDR Edition)

- **Board:** Standard layout, but custom properties (e.g., Memorable Locations, Date Ideas).
- **Mechanic:** "Chance" and "Community Chest" tiles are strictly replaced by the Truth or Dare card deck.

5.3 Truth & Dare Engine

The system must ensure cards are not repeated frequently. The backend must shuffle the `truthDareData.js` arrays upon room creation and maintain a `drawnCards` index in the room state.

> **Data Structure Example:**
> ```
> {
>   id: "t_01",
>   type: "truth",
>   text: "Apa momen paling berkesan waktu kita pertama kali ketemu?"
> }
> ```

6. WebSocket Event Dictionary

The implementation must strictly adhere to these event names for robust frontend-backend integration.

Event Name	Direction	Payload (Example)	Description
`create_room`	Client → Server	`{ playerName: "Player1" }`	Requests a new room creation.
`room_created`	Server → Client	`{ roomId: "X9BQ", status: "waiting" }`	Returns the generated room code.

Event Name	Direction	Payload (Example)	Description
`join_room`	Client → Server	`{ roomId: "X9BQ", playerName: "Player2" }`	Attempts to join an existing room.
`game_state_sync`	Server → Client	`{ boardState: [...], turn: "P1" }`	Synchronizes the entire board state to both clients.
`action_move`	Client → Server	`{ pieceId: "P1_A", targetTile: 45 }`	Fired when a player completes a drag-and-drop or dice roll logic.

7. UI/UX & Quality Assurance Guidelines

- **Pixel-Perfect Tailwind CSS:** Layouts must not break on mobile devices. Use `h-screen w-screen` constraints safely to avoid scrolling where not intended (especially for game boards).
- **Framer Motion Integration:**
  - Use `<AnimatePresence>` for mounting/unmounting Modals (like the T/D cards).
  - Use `layoutId` for smooth transitions when game pieces move from one tile to another.
- **Error Handling:** The Socket.io client must implement auto-reconnection logic. If a player disconnects temporarily, the backend must hold the game state in memory for at least 5 minutes.